



CS 498: Fine-Tuning Techniques

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
dLoRA team, and many colleagues

- **Model parallelism: distribute model execution across machines**
 - {Data, Tensor, Pipeline} parallelism
- **Mixed-precision training may hurt quality**
 - Activation checkpointing lowers memory footprint but increases computation

Low-Rank Adaptation (LoRA)

Tuning and Serving Many LoRAs

Pannel Discussion

- **Full-parameter Fine-Tuning:** to accommodate new tasks/datasets
 - Updating all model parameters is resource-intensive
- **Parameter-Efficient Fine-Tuning (PEFT):** update a small subset of parameters
 - Tradeoff between efficiency and model quality
- **Low-rank Adaptation (LoRA)**
 - Pre-trained LLMs have a “**low intrinsic dimension**”
 - LLMs can still learn efficiently despite a low-dim reparametrization

$$\Phi' = \Phi_0 + \Delta\Phi$$

Pre-trained
model weights

- **Insight:** insert small trainable weight matrix (as the side path) while keeping the base model frozen

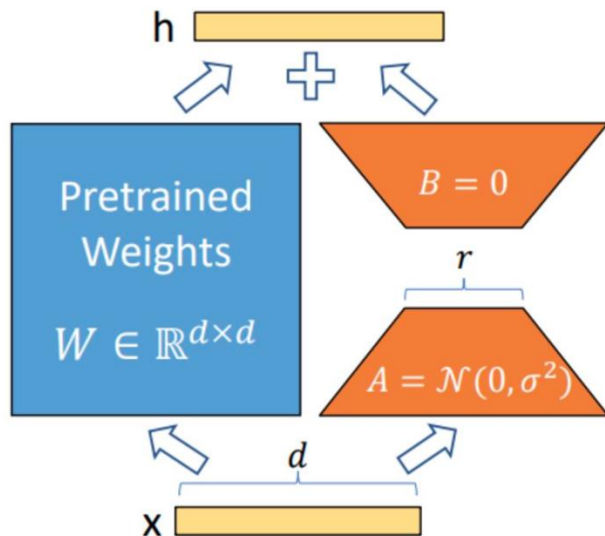


Figure 1: Our reparametrization. We only train A and B .

During training: for pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$, W_0 is fixed

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

$$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

- **Insight:** insert small trainable weight matrix (as the side path) while keeping the base model frozen

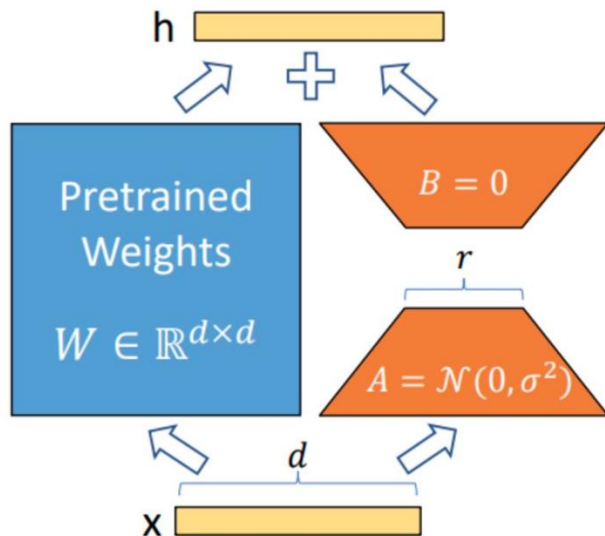


Figure 1: Our reparametrization. We only train A and B .

During training: for pre-trained weight $W_0 \in \mathbb{R}^{d \times k}$, W_0 is fixed

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

$$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

During inference:

$$W = W_0 + B A$$

- **Insight:** insert small trainable weight matrix (as the side path) while keeping the base model frozen

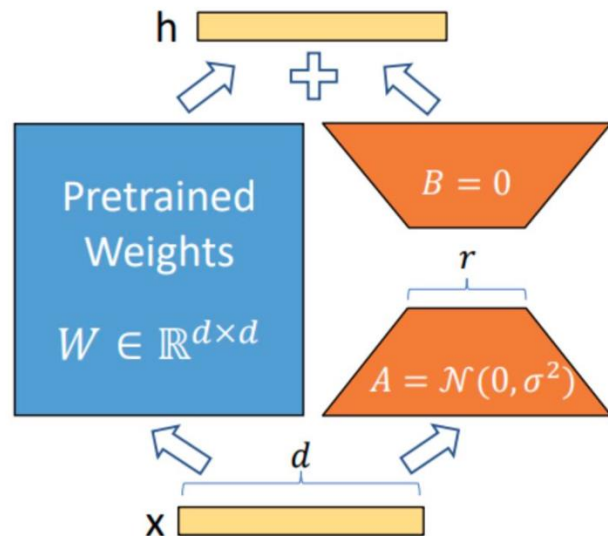


Figure 1: Our reparametrization. We only train A and B .

The gradients of the loss L with respect to A and B can be derived using the chain rule as follows:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial A} &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial A} \quad \text{constant} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + B A x)}{\partial A} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot B x^T\end{aligned}$$

W/o needing to compute the gradients for the base model (W_0)!

- **Insight:** insert small trainable weight matrix (as the side path) while keeping the base model frozen

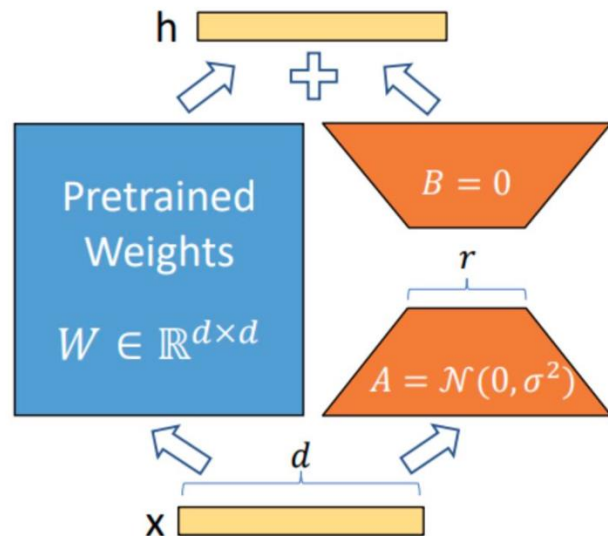


Figure 1: Our reparametrization. We only train A and B .

The gradients of the loss L with respect to A and B can be derived using the chain rule as follows:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial A} &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial A} \quad \text{constant} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + B A x)}{\partial A} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot B x^T\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial B} &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial B} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial (W_0 x + B A x)}{\partial B} \\ &= \frac{\partial \mathcal{L}}{\partial h} \cdot A x^T\end{aligned}$$

These partial derivatives are computed during the backward pass of backpropagation to update the parameters A and B accordingly.

Applying LoRA to Transformer

- In principle, LoRA can be applied to **any weight matrices in DL**
- LoRA's original paper only study changing the attention weights

$$W_q, W_k, W_v \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Applying LoRA to Transformer

- In principle, LoRA can be applied to **any weight matrices in DL**
- LoRA's original paper only study changing the attention weights

$$W_q, W_k, W_v \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

Benefits

- No need to track optimizer state for frozen params, smaller checkpoint size (GPT-3: 1.2TB [before] → 350GB [training] → 35MB [inference])
- Negligible inference latency as $r \ll d$
- speed up during training

LoRA-based Fine Tuning Performs Well



	# of Trainable Parameters = 18M					
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4
WikiSQL ($\pm 0.3\%$)	70.4	70.0	73.0	73.2	71.4	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.3\%$)	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q	68.8	69.6	70.5	70.4	70.0
MultiNLI ($\pm 0.1\%$)	W_q, W_v	91.3	91.4	91.3	91.7	91.4

What is the **optimal rank** for LoRA?

“low intrinsic dimension”

- increasing rank **does not cover more meaningful subspaces**, which suggests that a low-rank adaptation matrix is sufficient.

LoRA Model Selection Strategies

Language Models

Optimal LoRA Applications:

- Attention layers (W_q , W_v)
- Target rank: 8-64 depending on size
- Alpha: 16-32 for stable training

Vision Models

Optimal LoRA Applications:

- Convolutional layers
- Self-attention in Vision Transformers
- Target rank: 4-16 for efficiency

Multimodal Models

Optimal LoRA Applications:

- Cross-attention mechanisms
- Modality-specific projection layers
- Selective targeting based on task

LoRAs are mostly applied to the **attention** layer

LoRA Model Selection Strategies

Language Models

Optimal LoRA Applications:

- Attention layers (W_q , W_v)
- Target rank: 8-64 depending on size
- Alpha: 16-32 for stable training

Vision Models

Optimal LoRA Applications:

- Convolutional layers
- Self-attention in Vision Transformers
- Target rank: 4-16 for efficiency

Multimodal Models

Optimal LoRA Applications:

- Cross-attention mechanisms
- Modality-specific projection layers
- Selective targeting based on task

Selection Criteria

Key Considerations:

- Model architecture sensitivity
- Memory constraints & compute budget
- Task-specific adaptation requirements

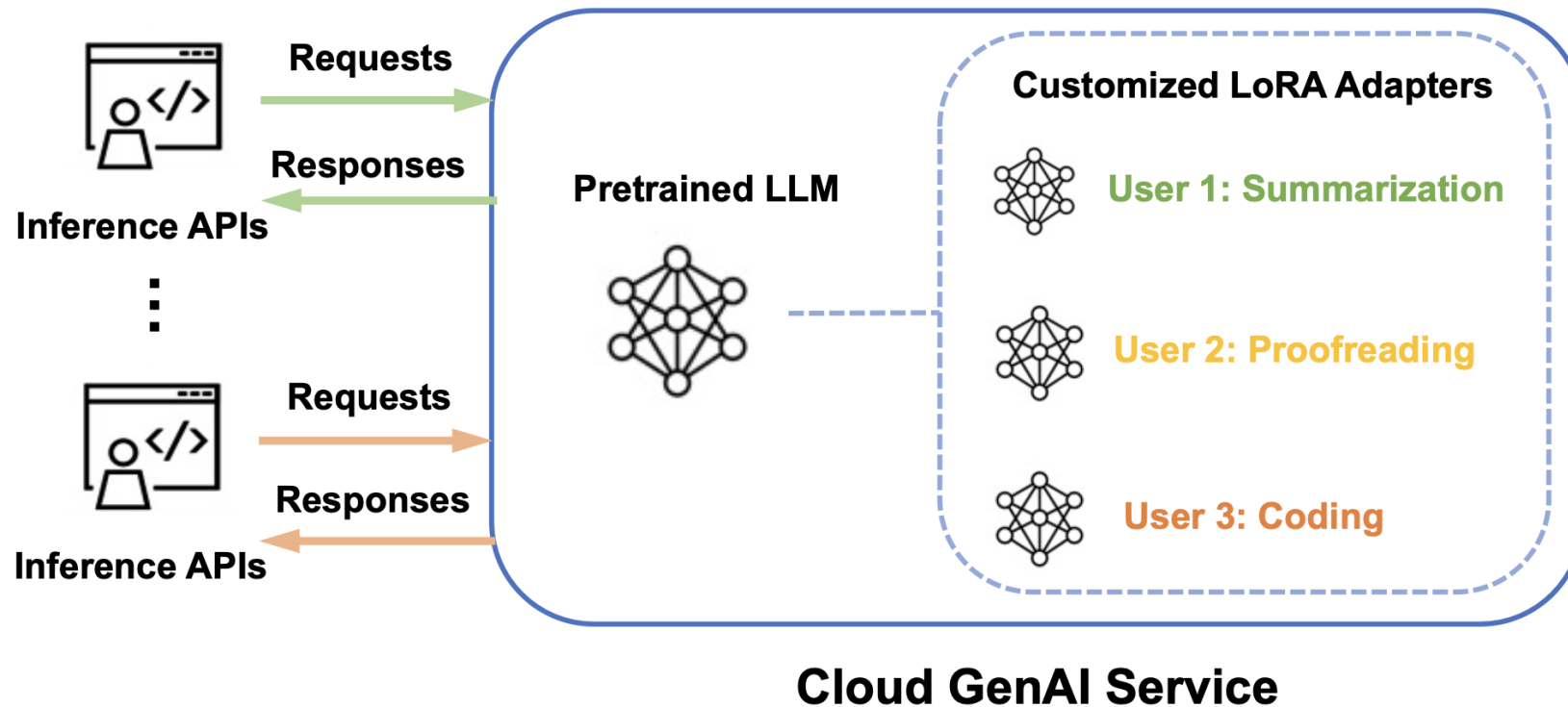
LoRAs are mostly applied to the **attention** layer

Low-Rank Adaptation (LoRA)

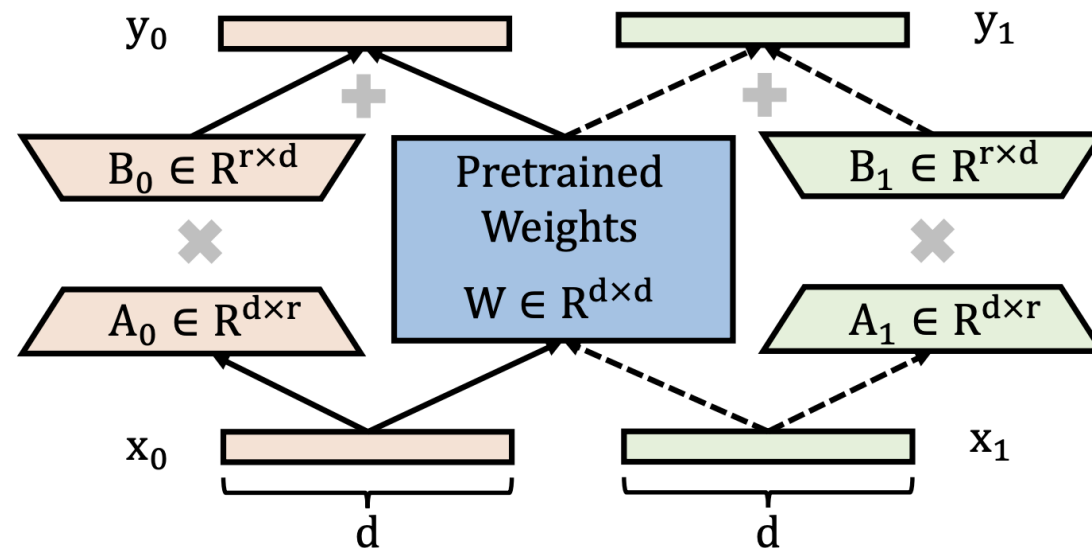
Tuning and Serving Many LoRAs

Pannel Discussion

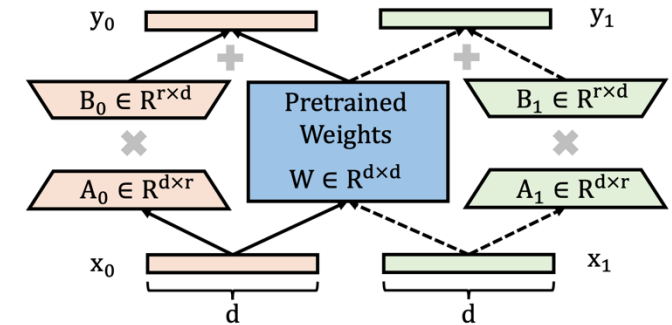
- **Cloud providers may host many adapters for an LLM**
 - Different users may use different adapters for different scenarios



- **Setting up model replicas for each user (LoRA) under-utilizes GPUs**
 - We can co-locate multiple LoRAs while sharing one base model

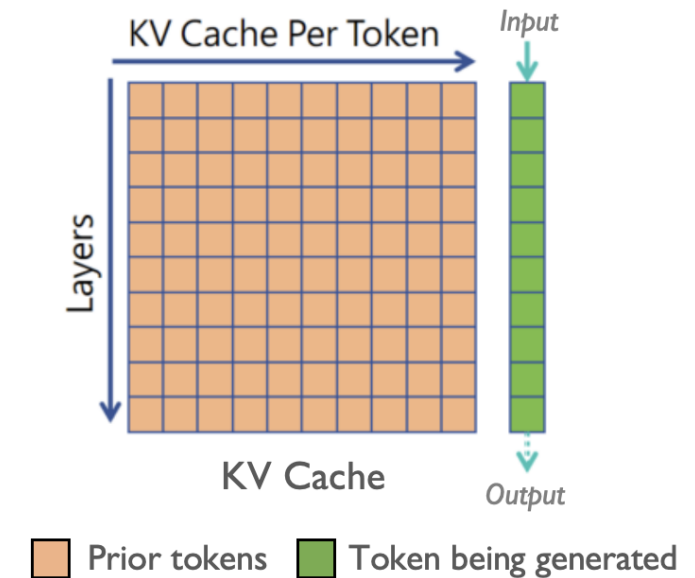


- **Store adapter weights in main memory (e.g., DRAM)**
 - GPU memory too small to store there
- **Issues:**
 - Dynamic fetching leads to memory fragmentation and I/O overhead
 - Hard to batch computation of adapters in non-contiguous memory

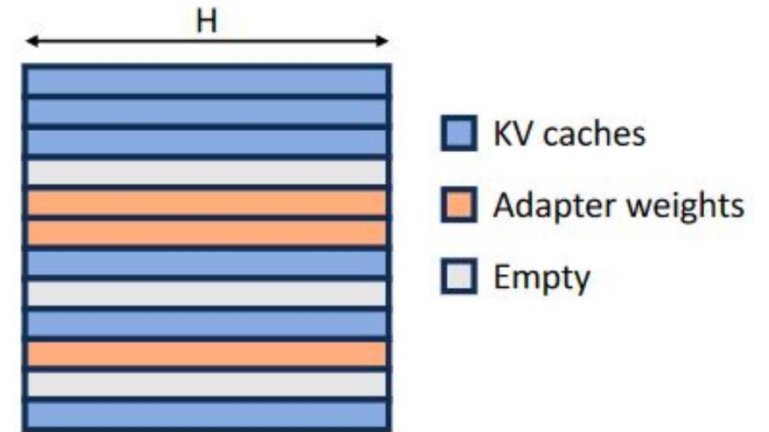


S-LoRA: Serving Thousand of Concurrent LoRA Adapters (MLSys'24)

- **Loading and unloading of active adapters**
 - Inactive adapters -> Main memory
 - Active adapters -> GPU memory
- **Naive loading and unloading problematic**
 - Causes memory fragmentation
 - Latency overhead in switching data



- **Loading and unloading of active adapters**
 - Inactive adapters -> Main memory
 - Active adapters -> GPU memory
- **Naive loading and unloading problematic**
 - Causes memory fragmentation
 - Latency overhead in switching data

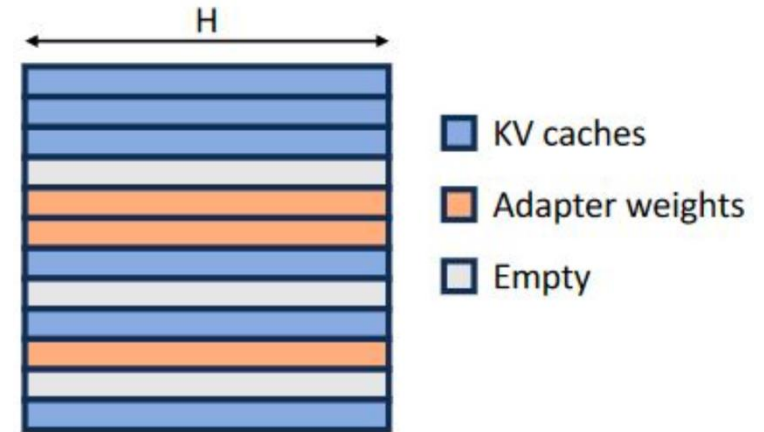


- **Loading and unloading of active adapters**

- Inactive adapters -> Main memory
- Active adapters -> GPU memory

- **Naive loading and unloading problematic**

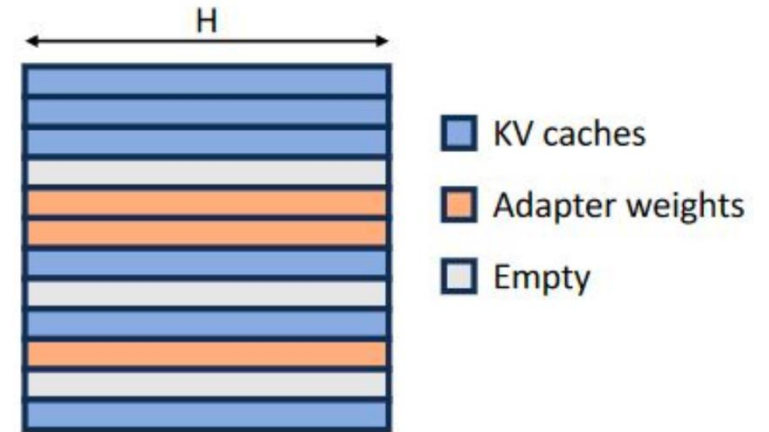
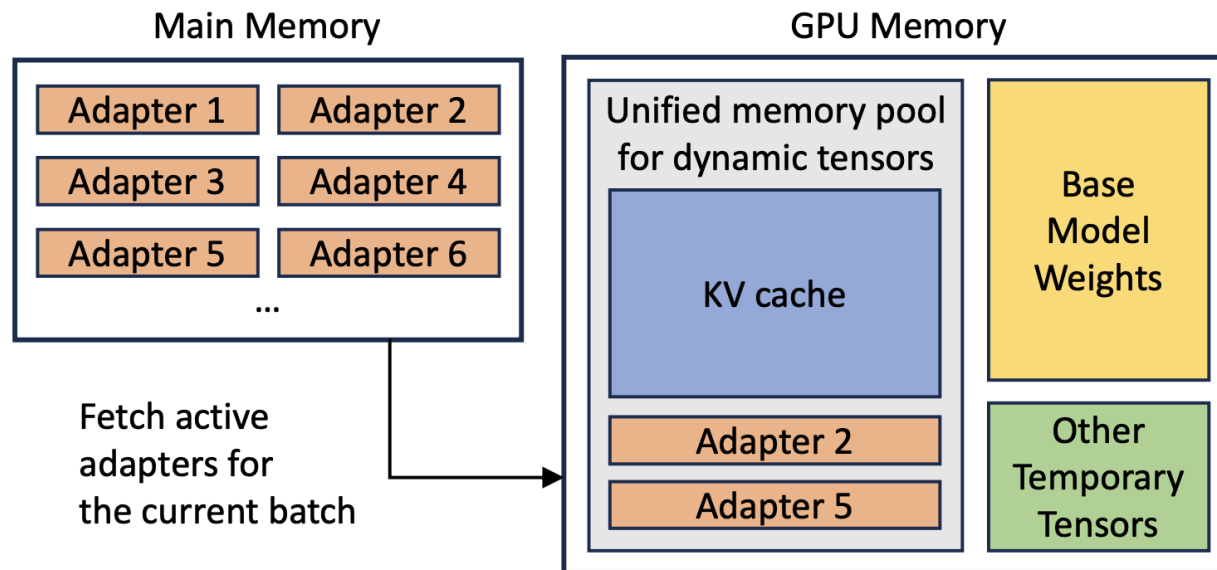
- Causes memory fragmentation
- Latency overhead in switching data



- **Unified paging: KV cache and adapter weights share same memory pool**

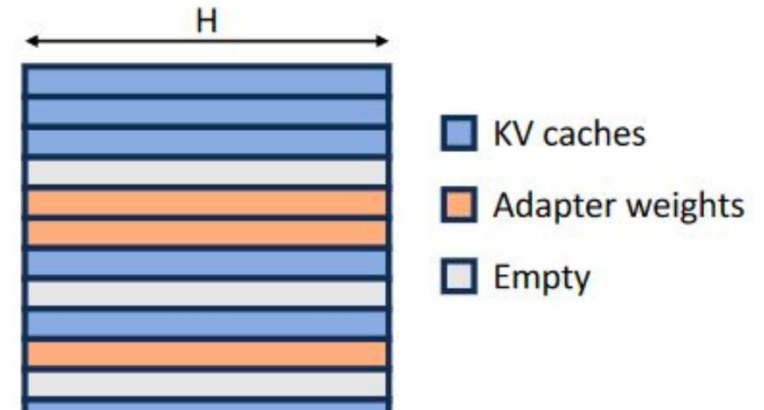
- Both KV cache and adapter weights allocated on request input and deallocated on request completion
- Interleaved storage to reduce fragmentation

- **Prefetching and overlapping**
 - Predict adapter weights needed for next batch when decoding current batch



- **Prefetching and overlapping**

- Predict adapter weights needed for next batch when decoding current batch



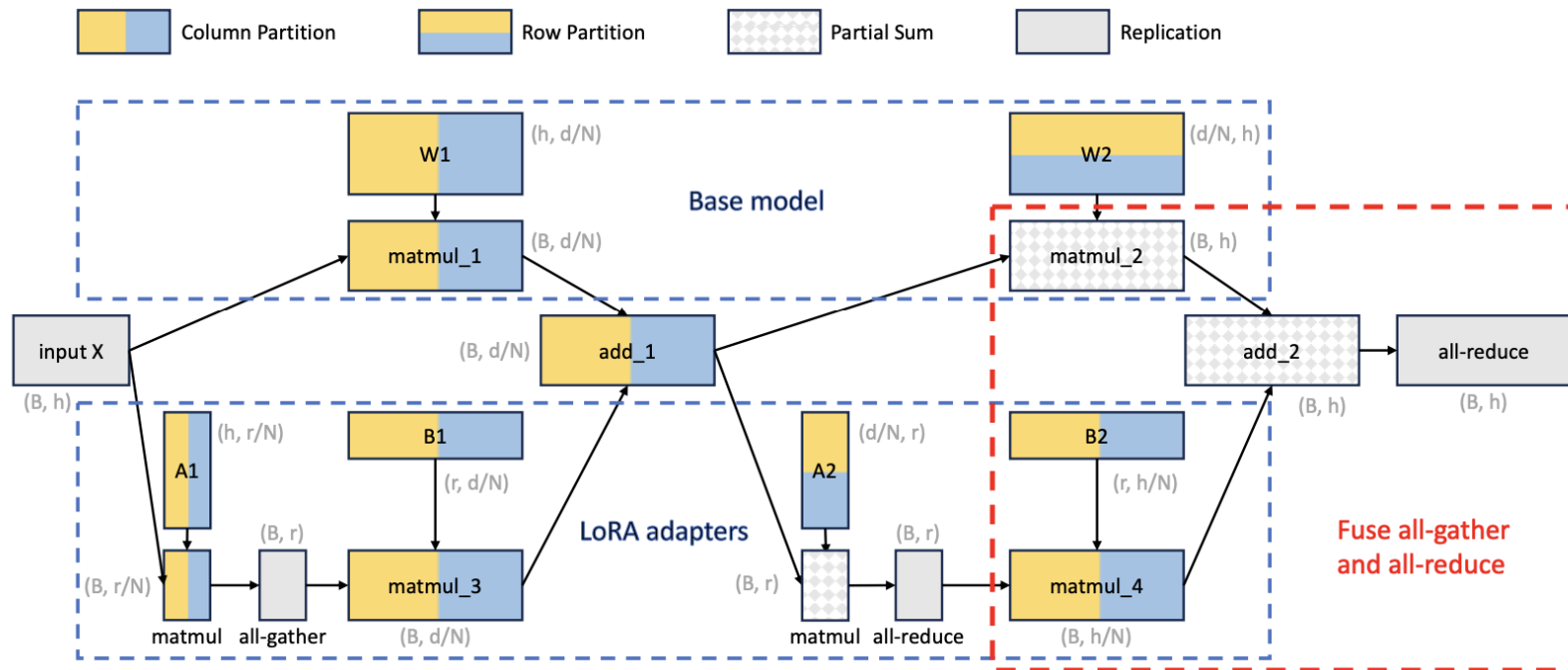
- **S-LoRA = Scalable LoRA**

- Unified Paging: Unified memory pool to reduce fragmentation, increase batch size
- Heterogenous Batching: Batching to allow concurrency of different adapters
- S-LoRA Tensor Parallelism: Split up adapter weights for inference across GPUs

Model Parallelism Supports S-LoRA



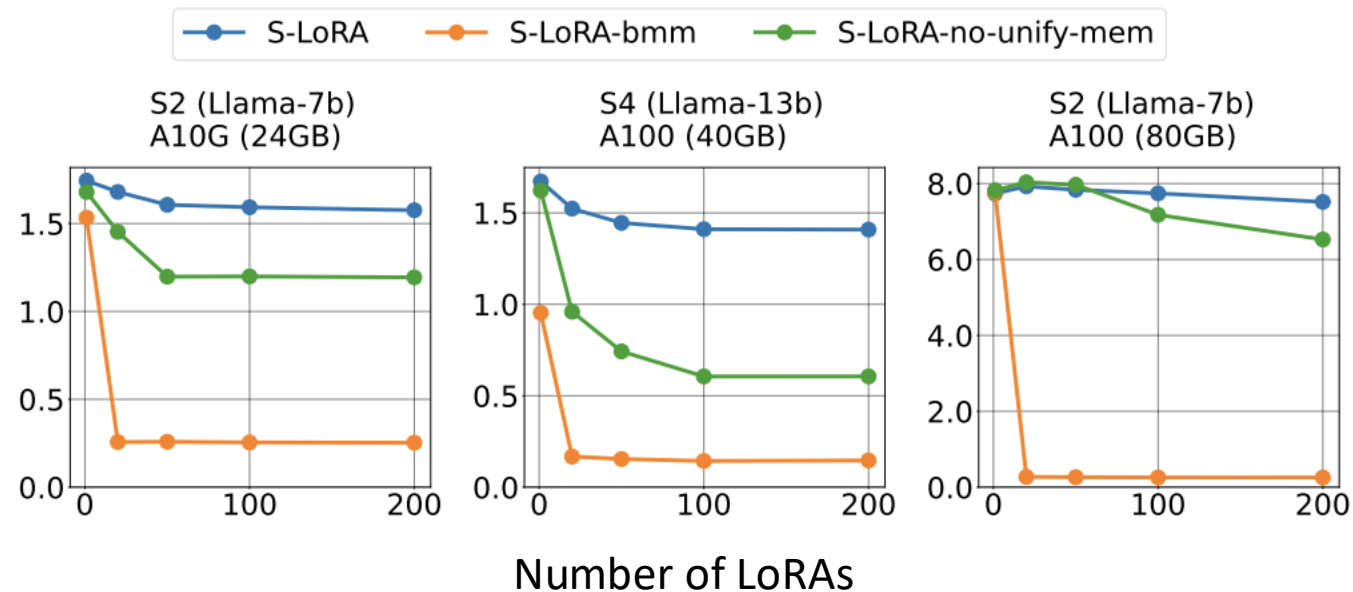
- **Models may be too large to fit into a single machine → model parallelism!**
 - Act very similar to single-model parallelism as LoRA $r \ll d$



S-LoRA Outperforms Naïve Serving without Co-locating LoRAs

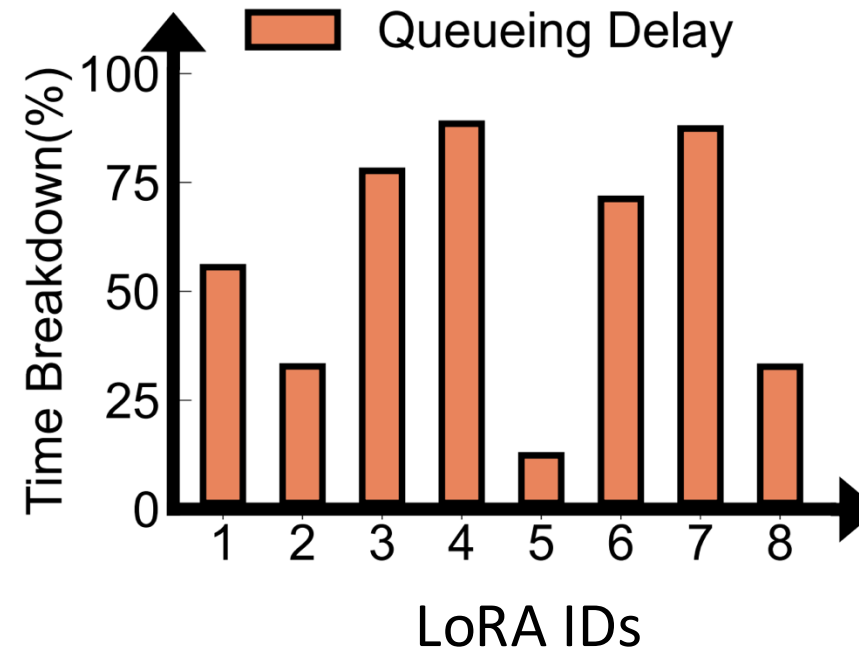


- **S-LORA-bmm**: S-LoRA without unified memory management and customized kernels
- **S-LORA-no-unify-mem**: S-LoRA without unified memory management



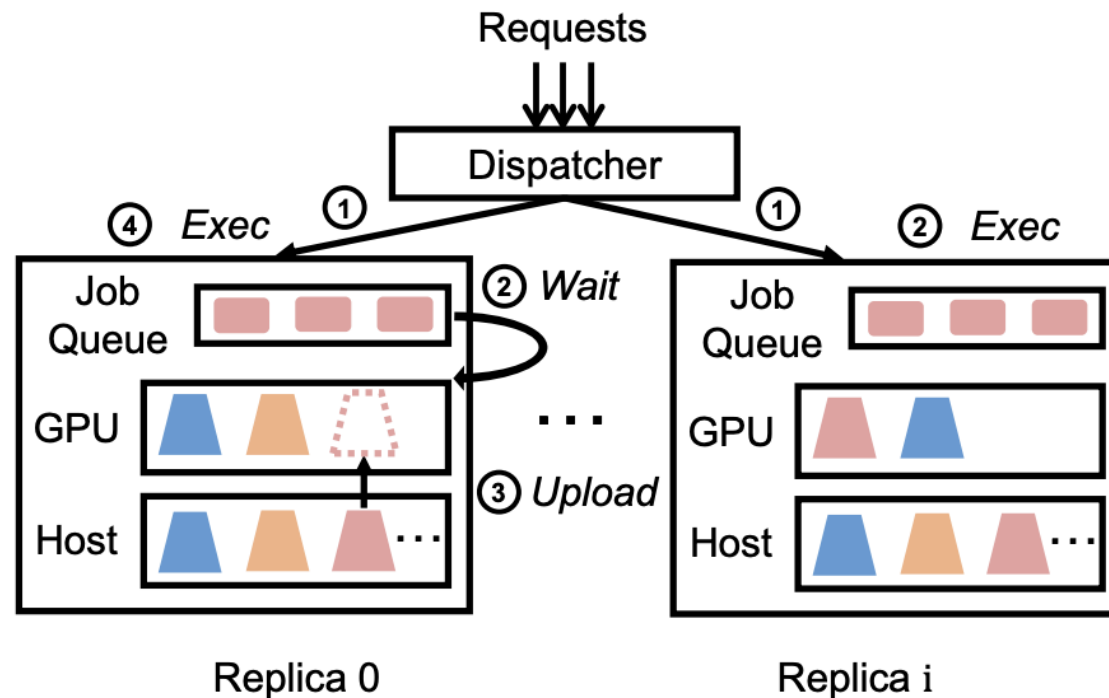
- **S-LoRA assumes all-or-nothing serving**
 - Use that LoRA or not

The burst of variable requests leads to severe **load imbalance** under static LoRA placement



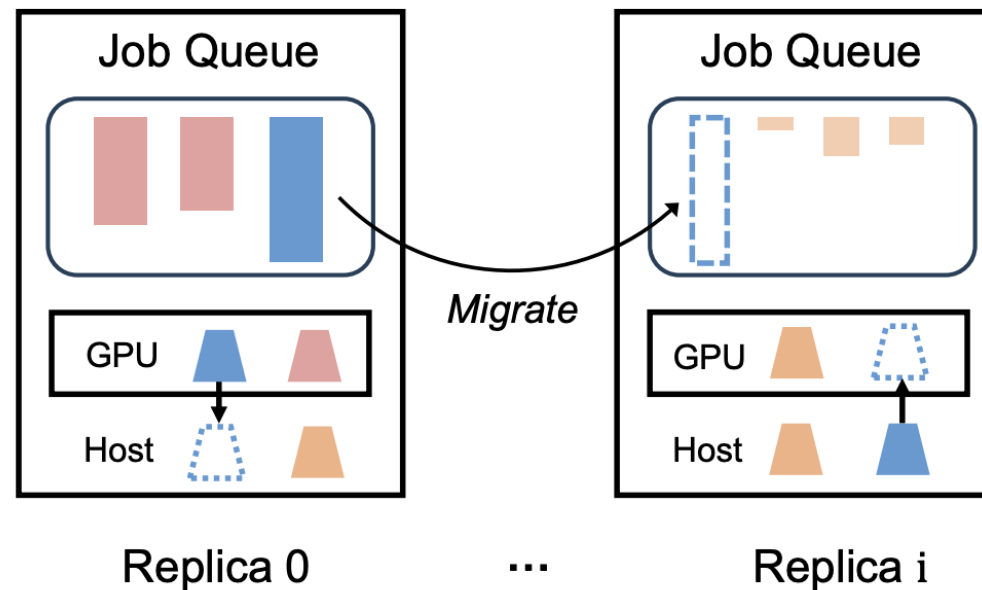
- **Key Insights**

- Set up multiple model replicas and replicate hot LoRAs
- Considering both adapter loading time and queuing delay to balance the load



- **Key Insights**

- Set up multiple model replicas and replicate hot LoRAs
- Considering both adapter loading time and queuing delay to balance the load
- Reactively migrate workload to balance the load



- **Parameter-Efficient Fine-Tuning (PEFT):** update a small subset of parameters
 - Low-Rank Adaptation (LoRA): insert small trainable weight matrix
 - Lower memory footprint and fine-tuning computation with little quality drop
- **Serving Many LoRAs:** practical deployments often host many LoRAs
 - S-LoRA: multiple LoRAs share the base model, offloading cold LoRAs to DRAM
 - dLoRA: S-LoRA + (reactive) load balance + (proactive) migration

Open problems?

(e.g., how to cache LoRA and KVCache)

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

Samyam Rajbhandari*, Jeff Rasley*, Olatunji Ruwase, Yuxiong He
`{samyamr, jerasley, olruwase, yuxhe}@microsoft.com`

ZeRO partitions optimizer states and model weights across machines to
reduce **peak memory demands**
at the cost of **higher communication**.